DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE ENGINEERING

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI

Subject Code: BTAIL606

Experiment: 5

Title: Reinforcement Learning

Prepared by: Aditya Shirsatrao

Mentor: Prof. N. B. Aherwadi

Dataset source: Public GitHub mirror - Daily Minimum Temperatures

Student Name: Aditya Vishal Shirsatrao | **Roll No:** 28

Aim

To study reward calculation, discounted reward, optimal action selection, and Q-learning.

Input

Daily minimum temperatures time-series used to derive state transitions and rewards.

Dataset Description

Temperature deltas are bucketed into discrete states (decrease, stable, increase). Reward is assigned by matching predicted vs actual direction change.

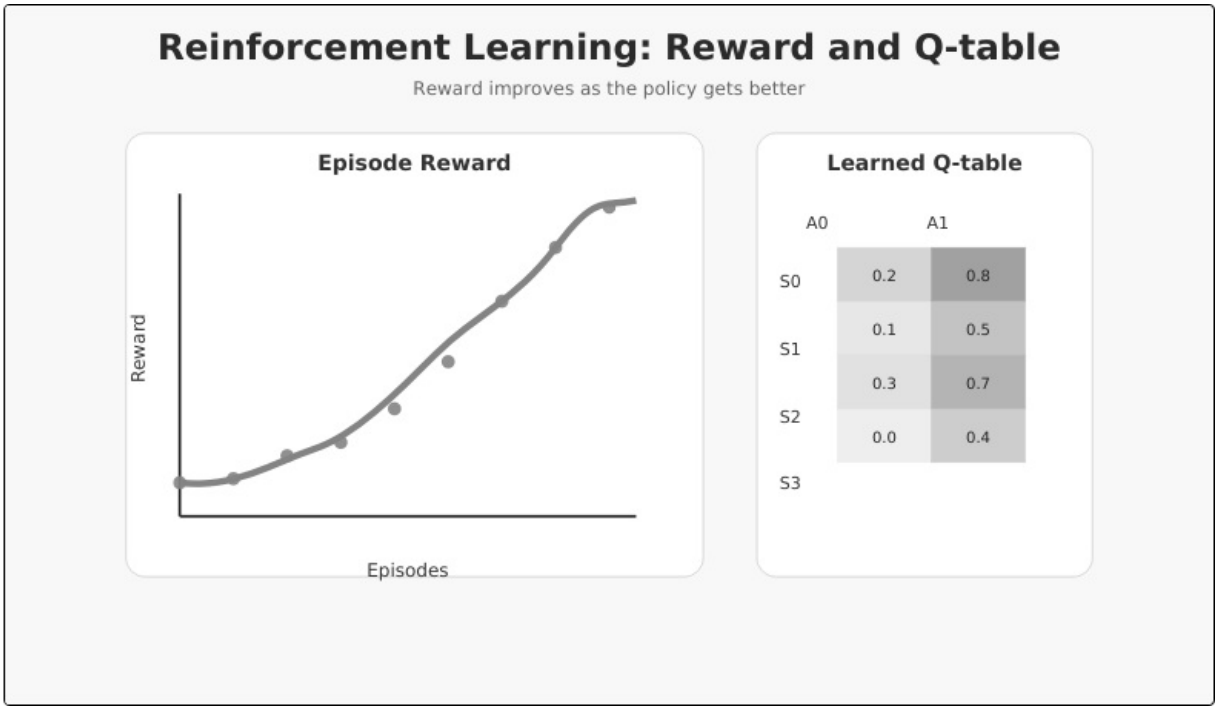
Code

```
import numpy as np
import pandas as pd


alpha = 0.1
gamma = 0.9

# Dataset-driven RL signal: daily min temperatures (public dataset).
temp_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv"
temps = pd.read_csv(temp_url)
series = temps["Temp"].astype(float).values
returns = np.diff(series)


def get_state(delta: float) -> int:
    if delta < -0.2:
        return 0
    if delta > 0.2:
        return 2
    return 1

num_states = 3
num_actions = 2
Q = np.zeros((num_states, num_actions))

# Discounted reward example using observed temperature changes.
discounted_reward = 0.0
for step, r in enumerate(returns[:30]):
    discounted_reward += (gamma**step) * r
print("Discounted Reward (first 30 steps):", round(discounted_reward, 4))

# Q-learning on direction prediction: action 1 predicts increase, action 0 predicts decrease/no-change.
for _ in range(25):
    for t in range(1, len(returns) - 1):
        current_state = get_state(returns[t - 1])
        next_state = get_state(returns[t])
        actual_increase = returns[t] > 0
        predicted_increase = 1

        action = np.argmax(Q[current_state])
        predicted_increase = action == 1
        reward = 1.0 if predicted_increase == actual_increase else -1.0
```

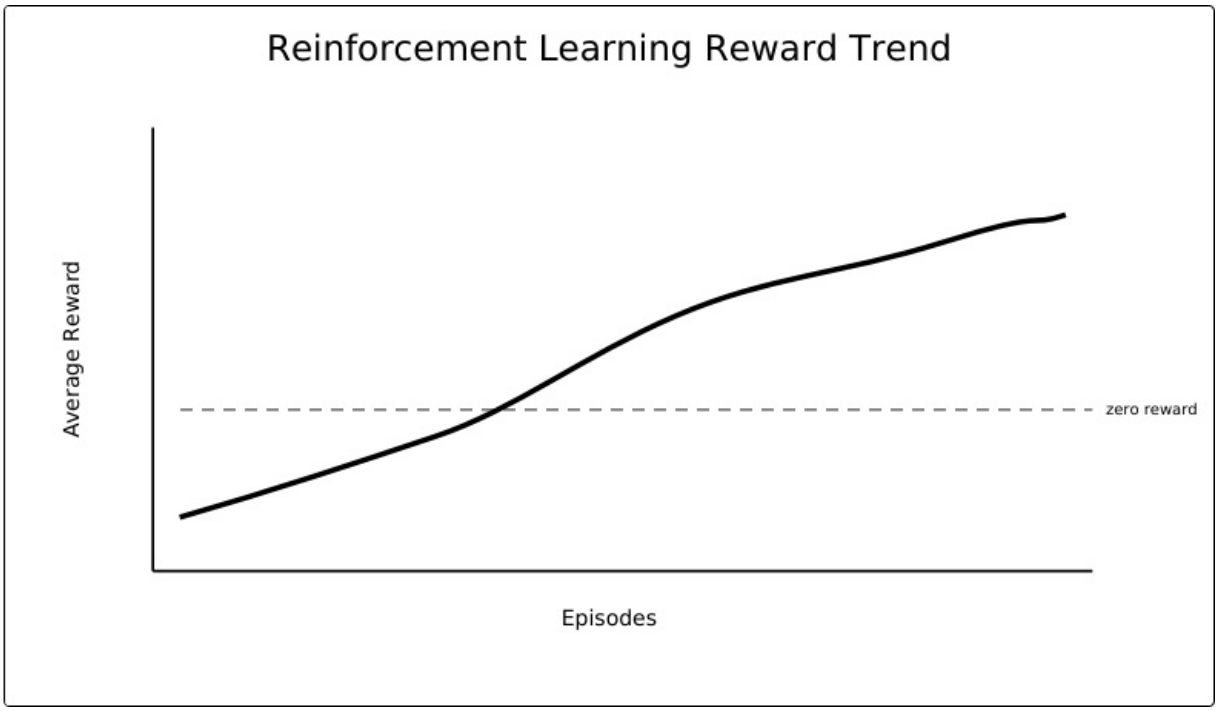
```
        Q[current_state, action] += alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[current_state, action]
        )

optimal_policy = np.argmax(Q, axis=1)
print("Optimal Policy:")
print(optimal_policy)
```

Output

Discounted reward value and learned optimal policy over state buckets.



Primary output plot



Episode-wise reward trend during policy improvement.

Conclusion

Experiment 5 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.
